

# 1 Editorial: Regate (Bridge Decomposition and Optimal Pairing)

## 1.1 Problem Overview

Given a graph with  $N$  vertices and  $M$  edges, each vertex has an associated value, and each edge has a cost. The problem involves finding the optimal way to pair vertices such that the total cost is minimized, with the key insight that bridges (edges whose removal disconnects the graph) play a crucial role in limiting which vertices can be paired together.

## 1.2 Key Observations

### 1.2.1 Bridges and 2-Edge-Connected Components

The fundamental observation is that any edge that is **not** a bridge lies inside a 2-edge-connected component. Within such a component, every vertex can reach every other vertex via multiple edge-disjoint paths. Consequently, any pairing of vertices within the same component is always achievable without crossing bridges.

If we contract each 2-edge-connected component into a single node and keep only the bridges, we obtain a tree structure. This tree is called the **bridge tree**.

- Vertices inside the same 2-edge-connected component can be paired arbitrarily (they don't require crossing any bridge).
- Vertices in different components can only be paired by using bridges between those components.

This decomposition allows us to decouple the problem: solve the internal pairings optimally within each component, then handle cross-component pairings through the bridge tree.

### 1.2.2 Optimal Pairing on a Tree

Consider the bridge tree as a rooted tree. For any edge (bridge) connecting two subtrees, if we decide that exactly  $k$  pairs need to cross this bridge, the contribution to the total cost is  $c \cdot k$ , where  $c$  is the cost of that bridge.

A greedy strategy works optimally: process bridges in **descending order of cost**. For each bridge:

- The only way to pair vertices across this bridge is through this specific bridge (since there's no alternative path).
- By processing expensive bridges first, we minimize their cost contribution while maximizing flexibility for cheaper bridges.

This greedy choice is optimal because bridges with higher costs should be used as little as possible, and any vertices that must cross a given bridge should be paired with the cheapest possible crossing.

## 1.3 Algorithm Description

The solution proceeds in four main phases:

### 1.3.1 Phase 1: Finding All Bridges

We perform a DFS from any vertex (vertex 1) computing:

- $lvl[v]$ : depth of vertex  $v$  in the DFS tree
- $lvlMin[v]$ : the minimum depth reachable from  $v$  via any edge (tree edge or back edge)

An edge  $(u, v)$  (where  $u$  is the parent of  $v$ ) is a bridge if and only if  $lvlMin[v] > lvl[u]$ , meaning  $v$  cannot reach any ancestor of  $u$  without using the edge  $(u, v)$ .

```
void getBridges(int nod, int parent) {
    for (edge& vec : g[nod])
        if (lvl[vec.x] == 0) {
            lvl[vec.x] = lvlMin[vec.x] = lvl[nod] + 1;
            getBridges(vec.x, nod);
            if (lvlMin[vec.x] == lvl[vec.x]) {
                isBridge[vec.pos] = true;
                bridges.push_back({nod, vec.x, vec.c});
            }
            lvlMin[nod] = min(lvlMin[nod], lvlMin[vec.x]);
        }
    else if (vec.x != parent) lvlMin[nod] = min(lvlMin[nod], lvl[vec.x]);
}
```

The running time is  $O(N + M)$ .

### 1.3.2 Phase 2: Forming Components

We run a DFS on the graph ignoring all bridge edges. Each connected component of this subgraph becomes a node in the bridge tree.

```
void dfs(int nod) {
    comp[nod] = nrcomp;
    cnt[nrcomp]++;
    for (edge& vec : g[nod])
        if (!isBridge[vec.pos] && !comp[vec.x]) dfs(vec.x);
}
```

After this phase, we have:

- $nrcomp$  components
- $comp[v]$ : the component ID of vertex  $v$
- $cnt[c]$ : the number of vertices in component  $c$

This phase also runs in  $O(N + M)$ .

### 1.3.3 Phase 3: Processing Bridges with DSU

We use a Disjoint Set Union (DSU) with additional data to track values during the greedy processing:

- $root[x]$ : parent in DSU
- $cnt[x]$ : size of component  $x$
- $val[x]$ : accumulated value in component  $x$

The bridges are sorted in descending order of cost. For each bridge between components  $x$  and  $y$  with cost  $c$ :

```
void unite(int x, int y, int cst) {
    x = getRoot(x);
    y = getRoot(y);
    if (x == y) return;
    if (cnt[x] > cnt[y]) swap(x, y);
    v[y].push_back(x);
    val[y] += 1LL * cst * cnt[x];
    val[x] += 1LL * cst * cnt[y] - val[y];
    root[x] = y;
    cnt[y] += cnt[x];
}
```

The key invariant maintained is that for any component root  $r$ :

$$val[r] = c_1 \cdot n_1 + c_2 \cdot n_2 + \dots$$

where each term represents the contribution of previously processed bridges connecting subcomponents.

When processing a node's own value (stored as a bridge with  $x = -1$ ), we compute:

$$ans[node] = c \cdot (cnt[root] - 1) - val[root]$$

This represents the contribution from pairing this node with all other nodes in its component at cost  $c$ .

After processing all bridges, we propagate values upward through the parent tree:

```
void propagate(int nod, int parent) {
    for (int& vec : v[nod])
        if (vec != parent) {
            val[vec] += val[nod];
            propagate(vec, nod);
        }
}
```

This phase runs in  $O((N + M) \cdot \alpha(N + M))$  due to DSU operations.

### 1.3.4 Phase 4: Computing Final Answers

For each vertex  $i$ , the final answer is:

$$answer[i] = ans[i] + val[where[i]]$$

where  $where[i]$  is the DSU root of vertex  $i$ 's component after all bridges are processed.

```
propagate(getRoot(1), 0);
for (int i = 1; i <= N; ++i) {
    ans[i] += val[where[i]];
    out << ans[i] << "\n";
}
```

## 1.4 Correctness Proof Sketch

1. **Lemma 1 (Bridge Decomposition):** After removing all bridges, each connected component contains vertices that can be paired arbitrarily without requiring bridge crossings.

**Proof:** If two vertices are in the same component after bridge removal, there exists a path between them using no bridges. Following this path, vertices can be matched step by step without crossing any bridge.  $\square$

2. **Lemma 2 (Bridge Tree Structure):** Contracting each component to a node and keeping only bridges yields a tree.

**Proof:** The original graph is connected (implied by the problem structure). Removing bridges disconnects it into components. Adding back only bridges creates a graph that is connected (bridges connect the components) and acyclic (adding a cycle would mean no edge in that cycle is a bridge). Hence, it's a tree.  $\square$

3. **Lemma 3 (Greedy Optimality):** Processing bridges in descending order of cost produces an optimal pairing.

**Proof:** Consider any optimal solution and any bridge  $e$  with cost  $c_e$ . Suppose  $k$  pairs cross  $e$  in the optimal solution. If  $e$  is not a bridge, this is impossible by definition. If  $e$  is a bridge, the vertices on each side must be paired across  $e$  to satisfy the pairing constraint.

When processing  $e$  in the greedy algorithm, all higher-cost bridges have already been processed. The greedy choice of pairing exactly  $k$  vertices across  $e$  (where  $k$  is the number that must cross, determined by component sizes) minimizes the contribution of  $e$  to the total cost. Any solution must incur at least  $c_e \cdot k$  from  $e$ , so the greedy choice is optimal for this bridge. By induction on the sorted order, the entire solution is optimal.  $\square$

4. **Lemma 4 (DSU Value Invariant):** After processing a set  $S$  of bridges, for each DSU component root  $r$ :

$$val[r] = \sum_{e \in S, \text{connects to } r} c_e \cdot n_e$$

where  $n_e$  is the size of the component on one side of bridge  $e$ .

**Proof:** The invariant holds initially (empty set,  $val = 0$ ). When processing a bridge connecting components  $x$  and  $y$  with cost  $c$ , the algorithm adds  $c \cdot cnt[x]$  to  $val[y]$  and adjusts  $val[x]$  accordingly. This exactly updates  $val$  to account for the new bridge's contribution.  $\square$

Combining these lemmas, the algorithm correctly computes the total cost for each vertex's contribution to the optimal pairing.

## 1.5 Complexity Analysis

- **Time Complexity:**  $O((N + M) \cdot \log^*(N + M) + (N + M) \cdot \log(N + M))$ 
  - Bridge finding DFS:  $O(N + M)$
  - Component DFS:  $O(N + M)$
  - Sorting bridges:  $O((N + M) \cdot \log(N + M))$
  - DSU operations:  $O((N + M) \cdot \alpha(N + M)) = O(N + M)$
- **Space Complexity:**  $O(N + M)$ 
  - Graph representation:  $O(N + M)$
  - Bridge storage:  $O(N + M)$
  - DSU arrays:  $O(N)$

Both bounds satisfy the problem constraints ( $N, M \leq 2 \times 10^5$ ).

## 1.6 Implementation Notes

- The input values for vertices are treated as bridges from a virtual node  $-1$  to allow uniform processing.
- The `where[]` array stores the initial representative for each vertex before DSU operations begin.
- The propagate function uses parent pointers to distribute accumulated values from parent components to children.
- Using `longlong` for all cost and value calculations is essential due to potential overflow with maximum constraints ( $c \leq 10^9$ ,  $n \leq 2 \times 10^5$ , products can exceed  $10^{14}$ ).
- The `swap` in `unite` ensures smaller components are merged into larger ones for balance, though this is not required for correctness (only for optimization).